

Datový typ pole

Příprava k maturitě - SPŠT IT

Obsah

1	Jednorozměrné pole	2
1.1	Vlastnosti pole	2
1.2	Výhody polí	2
1.3	Nevýhody polí	2
2	Deklarace a inicializace pole	2
2.1	C++ - pole	2
2.2	C# - pole	3
3	Přístup k jednotlivým prvkům pole	4
3.1	C++ - přístup k prvkům	4
3.2	C# - přístup k prvkům	4
3.3	Indexy od konce (C# 8.0+)	5
3.4	Praktické příklady	5
4	Pole jako parametr funkce	6
4.1	C++ - pole jako parametr	6
4.2	C# - pole jako parametr	7
5	Statické metody třídy Array (C#)	8
5.1	Řazení pole	8
5.2	Vyhledávání v poli	8
5.3	Kopírování pole	9
5.4	Vyplnění pole	9
5.5	Další užitečné metody	9
6	Instanční metody pole (C#)	10
6.1	Vlastnosti pole	10
6.2	Instance metody	10
6.3	LINQ metody pro pole	10
6.4	Příklady praktického použití	11
	Odpovědi na zadané podotázky	12
	Další materiály a zdroje	12

1 Jednorozměrné pole

Pole (array) je datová struktura, která uchovává **pevný počet** prvků **stejného datového typu** uspořádaných v **lineární posloupnosti**.

1.1 Vlastnosti pole

- **Pevná velikost** – po vytvoření nelze změnit
- **Homogenní** – všechny prvky mají stejný datový typ
- **Indexované** – přístup k prvkům pomocí indexu (číselné pozice)
- **Nultý index** – první prvek má index 0
- **Souvislá paměť** – prvky jsou uloženy vedle sebe v paměti

1.2 Výhody polí

1. Rychlý přístup k prvkům (konstantní čas $O(1)$)
2. Efektivní využití paměti
3. Jednoduché procházení prvků
4. Vhodné pro ukládání sekvencí dat

1.3 Nevýhody polí

- Pevná velikost – nelze dynamicky měnit
- Vkládání a mazání prvků je neefektivní
- Plýtvání pamětí, pokud není celé pole využito

2 Deklarace a inicializace pole

2.1 C++ – pole

Deklarace pole:

cpp

```
1 typPrvku nazevPole[velikost];
```

Příklady deklarace a inicializace:

cpp

```
1 // Deklarace bez inicializace (prvky mají náhodné hodnoty)
2 int cisla[5];
3
4 // Deklarace s inicializací – explicitní velikost
5 int cisla[5] = {10, 20, 30, 40, 50};
6
7 // Deklarace s inicializací – implicitní velikost
8 int cisla[] = {10, 20, 30, 40, 50}; // velikost 5
9
10 // Částečná inicializace (zbytek vyplněn nulami)
11 int cisla[5] = {10, 20}; // {10, 20, 0, 0, 0}
12
13 // Inicializace všech prvků na nulu
14 int cisla[5] = {}; // {0, 0, 0, 0, 0}
15
16 // Pole různých typů
```

```

17 double teploty[7];
18 char znaky[10];
19 bool stavy[3] = {true, false, true};
20 string jmena[4] = {"Jan", "Petra", "Karel", "Anna"};

```

Dynamické pole (C++):

cpp

```

1 // Dynamická alokace pomocí new
2 int velikost = 10;
3 int* pole = new int[velikost];
4
5 // Použití pole
6 pole[0] = 5;
7 pole[1] = 10;
8
9 // Uvolnění paměti
10 delete[] pole;

```

Zjištění velikosti pole:

cpp

```

1 int cisla[] = {1, 2, 3, 4, 5};
2 int velikost = sizeof(cisla) / sizeof(cisla[0]); // 5

```

2.2 C# – pole

Deklarace pole:

cs

```

1 typPrvku[] nazevPole;

```

Příklady deklarace a inicializace:

cs

```

1 // Deklarace bez inicializace
2 int[] cisla;
3
4 // Vytvoření pole s danou velikostí (prvky inicializovány na 0)
5 int[] cisla = new int[5]; // {0, 0, 0, 0, 0}
6
7 // Deklarace a inicializace s hodnotami
8 int[] cisla = new int[] {10, 20, 30, 40, 50};
9
10 // Zkrácený zápis (typ lze vynechat)
11 int[] cisla = {10, 20, 30, 40, 50};
12
13 // Pole různých typů
14 double[] teploty = new double[7];
15 char[] znaky = {'A', 'B', 'C'};
16 bool[] stavy = {true, false, true};
17 string[] jmena = {"Jan", "Petra", "Karel", "Anna"};
18
19 // Implicitně typované pole
20 var cisla = new[] {1, 2, 3, 4, 5}; // typ int[]

```

Zjištění velikosti pole:

cs

```
1 int[] cisla = {1, 2, 3, 4, 5};
2 int velikost = cisla.Length; // 5
```

3 Přístup k jednotlivým prvkům pole

Prvky pole jsou indexované od 0. K prvkům přistupujeme pomocí hranatých závorek `[]`.

3.1 C++ – přístup k prvkům

cpp

```
1 int cisla[5] = {10, 20, 30, 40, 50};
2
3 // Čtení hodnot
4 int prvni = cisla[0];    // 10
5 int druhy = cisla[1];    // 20
6 int posledni = cisla[4]; // 50
7
8 // Zápis hodnot
9 cisla[0] = 100;
10 cisla[2] = 300;
11
12 // Výpis všech prvků pomocí cyklu for
13 for (int i = 0; i < 5; i++) {
14     cout << cisla[i] << " ";
15 }
16
17 // Range-based for loop (C++11)
18 for (int cislo : cisla) {
19     cout << cislo << " ";
20 }
21
22 // Pozor na přístup mimo rozsah (undefined behavior)
23 // int x = cisla[10]; // NEBEZPEČNÉ!
```

3.2 C# – přístup k prvkům

cs

```
1 int[] cisla = {10, 20, 30, 40, 50};
2
3 // Čtení hodnot
4 int prvni = cisla[0];    // 10
5 int druhy = cisla[1];    // 20
6 int posledni = cisla[4]; // 50
7
8 // Zápis hodnot
9 cisla[0] = 100;
10 cisla[2] = 300;
11
12 // Výpis všech prvků pomocí cyklu for
13 for (int i = 0; i < cisla.Length; i++) {
14     Console.Write(cisla[i] + " ");
15 }
16
```

```

17 // Foreach cyklus
18 foreach (int cislo in ciska) {
19     Console.WriteLine(cislo + " ");
20 }
21
22 // Přístup mimo rozsah vyvolá výjimku
23 // int x = ciska[10]; // IndexOutOfRangeException

```

3.3 Indexy od konce (C# 8.0+)

cs

```

1 int[] ciska = {10, 20, 30, 40, 50};
2
3 // Index od konce pomocí operátoru ^
4 int posledni = ciska[^1]; // 50 (poslední prvek)
5 int predposledni = ciska[^2]; // 40 (předposlední prvek)
6
7 // Rozsahy (ranges)
8 int[] cast = ciska[1..4]; // {20, 30, 40}
9 int[] prvniTri = ciska[..3]; // {10, 20, 30}
10 int[] posledniDva = ciska[^2..]; // {40, 50}

```

3.4 Praktické příklady

Součet prvků pole:

cpp

```

1 // C++
2 int soucet = 0;
3 for (int i = 0; i < velikost; i++) {
4     soucet += ciska[i];
5 }

```

cs

```

1 // C#
2 int soucet = 0;
3 foreach (int cislo in ciska) {
4     soucet += cislo;
5 }

```

Nalezení maxima:

cpp

```

1 // C++
2 int max = ciska[0];
3 for (int i = 1; i < velikost; i++) {
4     if (ciska[i] > max) {
5         max = ciska[i];
6     }
7 }

```

cs

```

1 // C#
2 int max = ciska[0];
3 for (int i = 1; i < ciska.Length; i++) {
4     if (ciska[i] > max) {
5         max = ciska[i];
6     }
7 }

```

7 }

4 Pole jako parametr funkce

4.1 C++ – pole jako parametr

Pole se v C++ předává jako **ukazatel** na první prvek. Velikost pole se obvykle předává jako samostatný parametr.

cpp

```

1 // Pole jako parametr – předává se ukazatel
2 void vypisPole(int pole[], int velikost) {
3     for (int i = 0; i < velikost; i++) {
4         cout << pole[i] << " ";
5     }
6     cout << endl;
7 }
8
9 // Alternativní zápis s ukazatelem
10 void vypisPole(int* pole, int velikost) {
11     for (int i = 0; i < velikost; i++) {
12         cout << pole[i] << " ";
13     }
14     cout << endl;
15 }
16
17 // Změna hodnot v poli
18 void nasobDvema(int pole[], int velikost) {
19     for (int i = 0; i < velikost; i++) {
20         pole[i] *= 2; // změni původní pole
21     }
22 }
23
24 // Konstantní pole (nelze měnit)
25 void vypisKonstantniPole(const int pole[], int velikost) {
26     for (int i = 0; i < velikost; i++) {
27         cout << pole[i] << " ";
28         // pole[i] = 10; // CHYBA – nelze měnit
29     }
30 }
31
32 // Použití
33 int main() {
34     int cisla[] = {1, 2, 3, 4, 5};
35     int velikost = sizeof(cisla) / sizeof(cisla[0]);
36
37     vypisPole(cisla, velikost);
38     nasobDvema(cisla, velikost);
39     vypisPole(cisla, velikost); // {2, 4, 6, 8, 10}
40
41     return 0;
42 }

```

Vrácení pole z funkce:

cpp

```

1 // Dynamicky alokované pole
2 int* vytvorPole(int velikost) {
3     int* pole = new int[velikost];
4     for (int i = 0; i < velikost; i++) {
5         pole[i] = i + 1;
6     }
7     return pole;
8 }
9
10 int main() {
11     int* pole = vytvorPole(5);
12     // použití pole...
13     delete[] pole; // nutné uvolnit paměť!
14     return 0;
15 }

```

4.2 C# – pole jako parametr

V C# se pole předává jako **reference**, což znamená, že změny v metodě ovlivní původní pole.

cs

```

1 // Pole jako parametr
2 static void VypisPole(int[] pole) {
3     foreach (int cislo in pole) {
4         Console.Write(cislo + " ");
5     }
6     Console.WriteLine();
7 }
8
9 // Změna hodnot v poli
10 static void NasobDvema(int[] pole) {
11     for (int i = 0; i < pole.Length; i++) {
12         pole[i] *= 2; // změni původní pole
13     }
14 }
15
16 // Pole s params – proměnný počet argumentů
17 static int Soucet(params int[] cisla) {
18     int vysledek = 0;
19     foreach (int cislo in cisla) {
20         vysledek += cislo;
21     }
22     return vysledek;
23 }
24
25 // Vrácení pole z metody
26 static int[] VytvorPole(int velikost) {
27     int[] pole = new int[velikost];
28     for (int i = 0; i < velikost; i++) {
29         pole[i] = i + 1;
30     }
31     return pole;
32 }

```

```

33
34 // Použití
35 static void Main() {
36     int[] cisla = {1, 2, 3, 4, 5};
37
38     VypisPole(cisla);           // 1 2 3 4 5
39     NasobDvema(cisla);
40     VypisPole(cisla);           // 2 4 6 8 10
41
42     // params – různé způsoby volání
43     Console.WriteLine(Soucet(1, 2, 3));           // 6
44     Console.WriteLine(Soucet(10, 20, 30, 40));    // 100
45
46     // Vrácení pole
47     int[] novePole = VytvorPole(5);
48     VypisPole(novePole);           // 1 2 3 4 5
49 }

```

5 Statické metody třídy Array (C#)

Třída `System.Array` poskytuje statické metody pro práci s poli.

5.1 Řazení pole

CS

```

1 int[] cisla = {5, 2, 8, 1, 9};
2
3 // Seřazení vzestupně
4 Array.Sort(cisla); // {1, 2, 5, 8, 9}
5
6 // Obrácení pořadí
7 Array.Reverse(cisla); // {9, 8, 5, 2, 1}
8
9 // Řazení stringů
10 string[] jmena = {"Karel", "Anna", "Petr", "Eva"};
11 Array.Sort(jmena); // {Anna, Eva, Karel, Petr}

```

5.2 Vyhledávání v poli

CS

```

1 int[] cisla = {1, 2, 5, 8, 9};
2
3 // Binární vyhledávání (pole musí být seřazené!)
4 int index = Array.BinarySearch(cisla, 5); // 2
5
6 if (index >= 0) {
7     Console.WriteLine($"Nalezeno na indexu {index}");
8 } else {
9     Console.WriteLine("Nenalezeno");
10 }
11
12 // IndexOf – najde první výskyt
13 int[] cisla2 = {10, 20, 30, 20, 40};
14 int pozice = Array.IndexOf(cisla2, 20); // 1

```



```

15
16 // LastIndexOf – najde poslední výskyt
17 int posledni = Array.LastIndexOf(cisla2, 20); // 3

```

5.3 Kopírování pole

CS

```

1 int[] puvodni = {1, 2, 3, 4, 5};
2
3 // Kopírování celého pole
4 int[] kopie = new int[5];
5 Array.Copy(puvodni, kopie, 5);
6
7 // Kopírování části pole
8 int[] cast = new int[3];
9 Array.Copy(puvodni, 1, cast, 0, 3); // {2, 3, 4}
10
11 // Clone – vytvoří mělkou kopii
12 int[] klon = (int[])puvodni.Clone();

```

5.4 Vyplnění pole

CS

```

1 int[] cisla = new int[5];
2
3 // Vyplnění celého pole jednou hodnotou
4 Array.Fill(cisla, 10); // {10, 10, 10, 10, 10}
5
6 // Vyplnění části pole
7 Array.Fill(cisla, 5, 1, 3); // {10, 5, 5, 5, 10}
8 //          hodnota, start, count

```

5.5 Další užitečné metody

CS

```

1 int[] cisla = {1, 2, 3, 4, 5};
2
3 // Vymazání pole (nastavení na výchozí hodnoty)
4 Array.Clear(cisla, 0, cisla.Length); // {0, 0, 0, 0, 0}
5
6 // Změna velikosti (vytvoří nové pole)
7 Array.Resize(ref cisla, 10); // rozšíří na 10 prvků
8
9 // Existence prvku
10 bool existuje = Array.Exists(cisla, x => x > 3); // true/false
11
12 // Najdi prvek dle podmínky
13 int[] cisla2 = {1, 2, 3, 4, 5};
14 int prvni = Array.Find(cisla2, x => x > 3); // 4
15 int[] vsechny = Array.FindAll(cisla2, x => x > 2); // {3, 4, 5}
16
17 // ForEach – provede akci pro každý prvek
18 Array.ForEach(cisla2, x => Console.Write(x + " "));

```

6 Instanční metody pole (C#)

Každé pole v C# má vlastnosti a metody zděděné od `System.Array`.

6.1 Vlastnosti pole

CS

```
1 int[] cisla = {10, 20, 30, 40, 50};
2
3 // Length – počet prvků
4 int pocet = cisla.Length; // 5
5
6 // Rank – počet dimenzí (1 pro jednorozměrné pole)
7 int dimenze = cisla.Rank; // 1
8
9 // IsReadOnly – zda je pole pouze pro čtení
10 bool readOnly = cisla.IsReadOnly; // false
11
12 // IsFixedSize – zda má pole pevnou velikost
13 bool fixed = cisla.IsFixedSize; // true
```

6.2 Instance metody

CS

```
1 int[] cisla = {1, 2, 3, 4, 5};
2
3 // GetLength – délka v dané dimenzi
4 int delka = cisla.GetLength(0); // 5
5
6 // GetValue – získání hodnoty na indexu (jako object)
7 object hodnota = cisla.GetValue(2); // 3
8
9 // SetValue – nastavení hodnoty na indexu
10 cisla.SetValue(100, 2); // cisla[2] = 100
11
12 // Clone – vytvoří kopii pole
13 int[] kopie = (int[])cisla.Clone();
14
15 // CopyTo – zkopíruje do jiného pole
16 int[] cil = new int[10];
17 cisla.CopyTo(cil, 2); // zkopíruje od indexu 2
```

6.3 LINQ metody pro pole

Pole v C# podporují LINQ dotazy:

CS

```
1 using System.Linq;
2
3 int[] cisla = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
4
5 // Where – filtrování
6 int[] suda = cisla.Where(x => x % 2 == 0).ToArray(); // {2, 4, 6, 8, 10}
7
8 // Select – transformace
9 int[] druhe = cisla.Select(x => x * x).ToArray(); // {1, 4, 9, 16, ...}
```

```

10
11 // Sum – součet
12 int soucet = ciska.Sum(); // 55
13
14 // Average – průměr
15 double prumer = ciska.Average(); // 5.5
16
17 // Min / Max
18 int min = ciska.Min(); // 1
19 int max = ciska.Max(); // 10
20
21 // Count – počet prvků splňujících podmínku
22 int pocetVetsichNezPet = ciska.Count(x => x > 5); // 5
23
24 // Any – existuje alespoň jeden prvek splňující podmínku
25 bool existujeSude = ciska.Any(x => x % 2 == 0); // true
26
27 // All – všechny prvky splňují podmínku
28 bool vsechnaKladna = ciska.All(x => x > 0); // true
29
30 // First / Last
31 int prvni = ciska.First(); // 1
32 int posledni = ciska.Last(); // 10
33
34 // FirstOrDefault – vrátí první nebo výchozí hodnotu
35 int prvniVetsiNezDeset = ciska.FirstOrDefault(x => x > 10); // 0
36
37 // OrderBy / OrderByDescending
38 int[] serazene = ciska.OrderByDescending(x => x).ToArray(); // {10, 9, 8, ...}
39
40 // Take / Skip
41 int[] prvnichPet = ciska.Take(5).ToArray(); // {1, 2, 3, 4, 5}
42 int[] bezPrvnichPeti = ciska.Skip(5).ToArray(); // {6, 7, 8, 9, 10}
43
44 // Distinct – unikátní hodnoty
45 int[] sOpakovanim = {1, 2, 2, 3, 3, 3, 4};
46 int[] unikatni = sOpakovanim.Distinct().ToArray(); // {1, 2, 3, 4}

```

6.4 Příklady praktického použití

Statistiky pole:

cs

```

1 int[] znamky = {1, 2, 1, 3, 2, 1, 4, 3, 2};
2
3 int pocet = znamky.Length; // 9
4 double prumer = znamky.Average(); // 2.11
5 int nejlepsi = znamky.Min(); // 1
6 int nejhorsi = znamky.Max(); // 4
7 int pocetJednicek = znamky.Count(z => z == 1); // 3
8
9 Console.WriteLine($"Počet známek: {pocet}");
10 Console.WriteLine($"Průměr: {prumer:F2}");
11 Console.WriteLine($"Nejlepší: {nejlepsi}");
12 Console.WriteLine($"Nejhorší: {nejhorsi}");

```

```
13 Console.WriteLine($"Počet jedniček: {pocetJednicek}");
```

Filtrace a transformace:

CS

```
1 string[] jmena = {"Anna", "Petr", "Karel", "Eva", "Martin"};
2
3 // Jména začínající na 'A' nebo 'E', převedená na velká písmena
4 var vysledek = jmena
5     .Where(j => j.StartsWith("A") || j.StartsWith("E"))
6     .Select(j => j.ToUpper())
7     .ToArray();
8
9 // vysledek: {"ANNA", "EVA"}
10
11 // Seřazená jména delší než 4 znaky
12 var dlouha = jmena
13     .Where(j => j.Length > 4)
14     .OrderBy(j => j)
15     .ToArray();
16
17 // dlouha: {"Karel", "Martin", "Petr"}
```

Odpovědi na zadané podotázky

Každá otázka na začátku odpovědi obsahuje odkaz na konkrétní kapitolu v zápise s proklikem

Další materiály a zdroje

Žádné další zdroje